UNITÀ 2 · PROGETTARE E UTILIZZARE WEB SERVICE · LEZIONE 2

La comunicazione con RPC

Chiamare un metodo che vive su un altro computer come se fosse qui. Un'idea semplice e potente, con qualche trappola nascosta.

5G TPSI Stub, marshalling, Java RMI, gRPC, JSON-RPC

PER COMINCIARE

Due programmi su due computer devono collaborare. Tu come li faresti parlare?

SONDAGGIO

- **A** Socket e un formato di messaggi inventato da me
- **B** Chiamando i metodi dell'altro come se fossero locali
- **C** Scrivendo e leggendo un file condiviso
- **D** Mettendo messaggi in una coda che l'altro legge

MENTI CHE CI HANNO PRECEDUTO

Xerox PARC, 1984

> **L'idea delle chiamate di procedura remota è piuttosto semplice.**

Andrew Birrell e **Bruce Nelson**, ricercatori di Xerox PARC, in *Implementing Remote Procedure Calls*, 1984

L'osservazione

Tutti i programmatori sanno usare una chiamata di procedura. Allora facciamo sembrare una chiamata anche la comunicazione in rete.

La difficoltà

Il resto dell'articolo, decine di pagine, spiega perché realizzarla bene non è semplice per niente: guasti, nomi, prestazioni.

COME FUNZIONA

Il trucco: due sostituti, uno per parte

Client
chiama `somma(2, 3)` come un metodo qualunque

→

Stub client
impacchetta nome e parametri in byte (**marshalling**)

→

Rete
il messaggio viaggia su TCP o UDP

→

Skeleton
spacchetta (**unmarshalling**) e chiama il metodo vero

→

Server
esegue `somma` e restituisce 5

La risposta fa il percorso al contrario: il client non vede né socket né byte.

> All'esame: questo schema si disegna come **diagramma di sequenza**, con una colonna per client, stub, rete, skeleton e server.

MARSHALLING

Trasformare i parametri in byte, e ritorno

- Client e server devono essere d'accordo sul **formato**: quanti byte per un int, come si scrive una stringa.
- **Ordine dei byte**: le CPU Intel e ARM lavorano in little-endian, in rete si usa per tradizione il big-endian.
- I **puntatori** e i riferimenti non hanno senso su un'altra macchina: si passa una **copia** del dato.
- Formati **binari** (compatti, veloci) o **testuali** (XML, JSON: leggibili, più grandi).

Passaggio per copia

In locale, se il metodo modifica l'oggetto ricevuto, il chiamante vede la modifica.

In remoto il server lavora su una **copia**: il chiamante non vede nulla, a meno che il valore non torni nella risposta.

Una delle differenze che la "trasparenza" non può nascondere.

AL COMPUTER · 15 minuti

Guarda i byte di una chiamata

```
import java.io.*;

public class Marshal {
  public static void main(String[] a) throws IOException {
    var buf = new ByteArrayOutputStream();
    var out = new DataOutputStream(buf);
    out.writeUTF("somma");   // nome del metodo
    out.writeInt(2);         // primo parametro
    out.writeInt(300);       // secondo parametro
    for (byte b : buf.toByteArray())
      System.out.printf("%02X ", b);
    System.out.println("\n" + buf.size() + " byte");
    var in = new DataInputStream(
        new ByteArrayInputStream(buf.toByteArray()));
    System.out.println(in.readUTF() + "(" + in.readInt()
        + ", " + in.readInt() + ")");
  }
}
```

1. Scrivi **Marshal.java** ed eseguilo con `java Marshal.java`.
2. Trova nei byte la lunghezza della stringa, le lettere e i due int.
3. Cambia 300 in -1: che byte ottieni?

Controllo: `00 05 73 6F 6D 6D 61 00 00 00 02 00 00 01 2C`, 15 byte, poi `somma(2, 300)`. Con -1 ottieni `FF FF FF FF`.

PUZZLE

Metti in ordine una chiamata remota

1. Lo stub fa il marshalling di nome e parametri
2. Il client riceve il valore di ritorno e riprende
3. La risposta torna in rete e lo stub la decodifica
4. Il server esegue e restituisce il risultato allo skeleton
5. Il client chiama il metodo sullo stub locale
6. Il messaggio viaggia in rete fino al server
7. Lo skeleton fa l'unmarshalling e chiama il metodo vero

QUIZ A TRABOCCHETTO

Il client chiama `versa(conto, 100)` e dopo 5 secondi scatta il timeout. Che cosa sa?

QUIZ

A Che il server non ha eseguito il versamento

B Che il server ha eseguito, ma la risposta si è persa

C Che la rete gli manderà un'eccezione con la causa precisa

D Niente di certo: richiesta persa, server lento o risposta persa

SEMANTICHE DI INVOCAZIONE

Se riprovo, quante volte viene eseguito?

Semantica	Come si ottiene	Rischio
Almeno una volta	Il client ritrasmette finché non riceve risposta	Esecuzioni doppie
Al più una volta	Il server ricorda gli ID già visti e scarta i duplicati	Può non essere eseguita mai
Esattamente una volta	Ritrasmissione + ID + memoria persistente sul server	Costosa, difficile da garantire

Idempotente

Eseguirla due volte dà lo stesso effetto di una: `leggiSaldo()`, `impostaEmail(x)`. Si può riprovare senza paura.

Non idempotente

`versa(100)` eseguita due volte versa 200. Si protegge con un **ID univoco** della richiesta.

METTITI ALLA PROVA

La risposta si è persa. Quale chiamata puoi ripetere senza danni?

QUIZ

A `creaOrdine(carrello)`

B `impostaIndirizzo(utente, "Via Roma 1")`

C `aggiungiPunti(utente, 10)`

D `inviaNotifica(utente, "Spedito")`

SINCRONA O ASINCRONA

Aspettare la risposta, oppure no

Sincrona

Il client si blocca finché arriva la risposta. Semplice da scrivere, ma un server lento blocca anche il client.

Java RMI, una chiamata HTTP classica

Asincrona

Il client riceve subito un **future** o registra una **callback** e continua a lavorare.

`sendAsync` di HttpClient, gli stub asincroni di gRPC

Senza risposta

Il client invia e non aspetta nulla (*one-way*). Utile per log e notifiche.

Non sa mai se è arrivata

```
var futuro = client.sendAsync(req, BodyHandlers.ofString());   // non si blocca
// ... intanto il programma fa altro ...
futuro.thenAccept(r -> System.out.println(r.body()));           // callback
```

JAVA RMI

RPC a oggetti, già dentro il JDK

- L'interfaccia remota estende `java.rmi.Remote`; ogni metodo dichiara `throws RemoteException`.
- Il server **esporta** l'oggetto: Java crea lo stub e apre una porta.
- Il **registro RMI** (porta 1099) associa un nome allo stub: è il servizio di **binding**.
- Il client fa `lookup("calc")`, riceve lo stub e chiama i metodi.
- Parametri e risultati devono essere `Serializable` o a loro volta remoti.

Pregi e limiti

+ nessuna libreria esterna, tipi Java veri, eccezioni.

− solo Java con Java: niente interoperabilità con altri linguaggi.

− passa male i firewall: porte dinamiche, protocollo proprietario.

Per questo sul Web hanno vinto i web service.

AL COMPUTER · 20 minuti

Un servizio RMI: interfaccia e server

```java
// ServerRmi.java
import java.rmi.registry.*;
import java.rmi.server.UnicastRemoteObject;
public class ServerRmi implements Calcolatrice {
  static Registry registro;      // da tenere vivo
  public int somma(int a, int b) {
    System.out.println("Chiamata remota: somma(" + a + ", " + b + ")");
    return a + b;
  }
  public static void main(String[] args) throws Exception {
    Calcolatrice stub = (Calcolatrice)
        UnicastRemoteObject.exportObject(new ServerRmi(), 0);
    registro = LocateRegistry.createRegistry(1099);
    registro.rebind("calc", stub);
    System.out.println("Server RMI pronto sulla porta 1099");
  }
}
```

```java
// Calcolatrice.java
import java.rmi.*;
public interface Calcolatrice
    extends Remote {
  int somma(int a, int b)
      throws RemoteException;
}
```

1. Cartella **rmi** con i due file, poi `javac *.java`
2. Avvia `java ServerRmi` e lascia aperta la finestra.

Controllo: compare **Server RMI pronto** e il programma resta in attesa di chiamate.

AL COMPUTER · 15 minuti

Il client RMI, e che cosa succede quando il server cade

```java
// ClientRmi.java
import java.rmi.registry.*;
public class ClientRmi {
  public static void main(String[] args) throws Exception {
    String host = args.length > 0 ? args[0] : "localhost";
    Registry registro = LocateRegistry.getRegistry(host, 1099);
    Calcolatrice calc = (Calcolatrice) registro.lookup("calc");
    System.out.println("Ho in mano: " + calc.getClass().getName());
    System.out.println("2 + 3 = " + calc.somma(2, 3));
  }
}
```

1. In un secondo terminale: `javac ClientRmi.java` e `java ClientRmi`
2. Guarda anche la finestra del server.
3. Ferma il server con Ctrl+C e rilancia il client.

Controllo: il client stampa una classe come `jdk.proxy1.$Proxy0` (lo stub!) e **2 + 3 = 5**. A server spento: `ConnectException`.

GRPC

L'RPC moderno: un contratto, tanti linguaggi

```
// calcolatrice.proto
syntax = "proto3";
service Calcolatrice {
  rpc Somma (Operandi) returns (Risultato);
}
message Operandi  { int32 a = 1; int32 b = 2; }
message Risultato { int32 valore = 1; }
```

- Creato da Google, open source dal 2015.
- Il file **.proto** è l'IDL: da qui si generano stub per Java, Python, Go, C#, Kotlin…
- Dati in **Protocol Buffers**, binario e compatto: `Operandi{a=2, b=300}` diventa `08 02 10 AC 02`, 5 byte.
- Viaggia su **HTTP/2**: più chiamate sulla stessa connessione, streaming nei due sensi.

PUZZLE

Abbina ogni tecnologia RPC alla sua carta d'identità

Sun RPC	---	Messaggi XML descritti da un WSDL
Java RMI	---	Messaggi JSON con method, params e id
SOAP	---	Anni '80, IDL in file .x, la usa ancora NFS
gRPC	---	File .proto, Protocol Buffers su HTTP/2
JSON-RPC 2.0	---	Solo tra programmi Java, registro sulla porta 1099

OGGI

Come un'AI chiama uno strumento

```
// l'app AI chiede al server MCP
{ "jsonrpc": "2.0", "id": 7,
  "method": "tools/call",
  "params": { "name": "meteo",
     "arguments": { "citta": "Milano" } } }

// il server MCP risponde
{ "jsonrpc": "2.0", "id": 7,
  "result": { "content": [
     { "type": "text", "text": "14 °C, nuvoloso" } ] } }
```



AL COMPUTER · 25 minuti

Un mini server JSON-RPC, provato con curl

```
var srv = HttpServer.create(new InetSocketAddress(8090), 0);
srv.createContext("/rpc", ex -> {
  String q = new String(ex.getRequestBody().readAllBytes());
  Matcher p = Pattern.compile("\\[(-?\\d+),\\s*(-?\\d+)\\]").matcher(q);
  Matcher id = Pattern.compile("\"id\":\\s*(\\d+)").matcher(q);
  String n = id.find() ? id.group(1) : "null";
  String r = q.contains("\"somma\"") && p.find()
    ? "\"result\":" + (Integer.parseInt(p.group(1)) + Integer.parseInt(p.group(2)))
    : "\"error\":{\"code\":-32601,\"message\":\"Method not found\"}";
  byte[] out = ("{\"jsonrpc\":\"2.0\"," + r + ",\"id\":" + n + "}").getBytes();
  ex.getResponseHeaders().set("Content-Type", "application/json");
  ex.sendResponseHeaders(200, out.length);
  ex.getResponseBody().write(out);
  ex.close();
});
srv.start();
```

1. **JsonRpc.java**: codice nel `main`; import di HttpServer, InetSocketAddress e java.util.regex.
2. **richiesta.json**: chiama **somma** con params [2, 40] e id 1.
3. `curl -d @richiesta.json http://localhost:8090/rpc`
4. Prova un metodo che non esiste.

Controllo: `{"jsonrpc":"2.0","result":42,"id":1}`. Con un metodo sconosciuto: `"error":{"code":-32601,…}`, il codice che JSON-RPC 2.0 riserva a "Method not found".

BIGLIETTO D'USCITA

Completa prima di uscire

Il componente che, sul client, finge di essere l'oggetto remoto si chiama _____.

Trasformare nome e parametri in byte da spedire si dice _____.

Un'operazione che ripetuta due volte ha lo stesso effetto di una è _____.

In Java RMI il registro ascolta di solito sulla porta _____.

Il Model Context Protocol usa messaggi _____ 2.0.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- RPC: una chiamata remota travestita da locale
- Stub e skeleton fanno marshalling e unmarshalling
- Timeout: il client non sa se il server ha eseguito
- Idempotenza e ID di richiesta per riprovare in sicurezza
- Java RMI: solo Java; gRPC: .proto, HTTP/2, multi-linguaggio
- JSON-RPC 2.0, il formato dei messaggi di MCP

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.